

Arquitetura de Fine-tuning Local e Aprendizado

Como o Forge aprende, o que é salvo, como o treino vira um adapter LoRA e como isso entra na execução local.

Status	Fine-tuning real via LoRA local: implementado como base funcional. Não é full retraining do GGUF base.
Escopo	Workspace local do cliente, Knowledge Base, trajetórias verificadas, training packs, llama.cpp finetune, adapter registry e runtime Forge.
Gerado em	14/06/2026 21:44
Arquivos-chave	src/main/services/kb-quality.ts; src/main/services/kb-learning.ts; src/main/services/forge-local-training.ts; src/main/services/smart-exec/llama-runtime.ts; src/main/services/smart-exec/orchestrator.ts; src/renderer/src/pages/KnowledgePage.tsx

Resposta curta

Sim: agora existe um pipeline de fine-tuning local de verdade, no sentido técnico de pós-treino supervisionado com LoRA adapter. O modelo base não é regravado; o Forge carrega o GGUF base + adapter treinado por workspace. A etapa que ainda precisa virar gate obrigatório é avaliação automática pós-treino antes de ativar um adapter novo.

1. O que significa “fine-tuning” aqui

No Orkestral, “fine-tuning” não significa apenas salvar memórias em uma base de conhecimento. O sistema agora tem um caminho para transformar execuções verificadas em exemplos de treino, exportar esses exemplos em formato de trajetória e treinar localmente um adapter LoRA para o modelo Forge.

Esse adapter é carregado junto com o modelo GGUF local durante a inferência. Na prática, o modelo passa a receber uma influência aprendida a partir das boas execuções daquele workspace: estilo de patch, padrões do projeto, convenções de arquitetura e decisões recorrentes que passaram pelos filtros de qualidade.

É fine-tuning?	Sim, por adapter LoRA local.
É RAG?	Não. RAG continua existindo, mas é consulta em tempo de execução. LoRA é pós-treino persistido.
Regrava o modelo base?	Não. O GGUF base permanece igual; o aprendizado

	fica no adapter lora.bin.
É seguro para cliente?	Mais seguro do que treinar um modelo global, porque o adapter é por workspace e fica local.

2. Arquitetura ponta a ponta

O fluxo completo tem sete fases. Cada fase tem uma responsabilidade clara para evitar que “qualquer log” vire treino e para manter o modelo local útil sem contaminar o conhecimento com ruído.

Fase	Nome	Função
1	Execução da tarefa	Agente/CLI/Forge executa uma issue, usa ferramentas, edita arquivos, roda validações e gera eventos.
2	Registro de trajetória	A execução vira uma trajetória: objetivo, agente, fonte/repo, arquivos tocados, ferramentas, resultado, verificação e metadados.
3	Curadoria intrínseca	O sistema calcula qualidade/relevância. Execuções fracas, repetidas, sem escopo ou desfeitas por undo são rejeitadas.
4	Persistência como exemplo	Exemplos aprovados entram em ai_training_examples e podem alimentar training packs.
5	Exportação do pack	O Knowledge Center exporta train.trajectory.jsonl, validation.trajectory.jsonl e rejected.trajectory.jsonl.
6	Treino local	forge:start-training chama llama.cpp finetune e gera lora.bin dentro do workspace.
7	Ativação no runtime	O adapter aprovado é registrado em forge-adapters.json e injetado no node-llama-cpp via loraPath.

3. Técnicas utilizadas

Técnica	Onde entra	Por que importa
RAG semântico	Busca na Knowledge Base e embeddings.	Dá contexto fresco do repo sem precisar treinar tudo no peso do modelo.
Curadoria de trajetórias	kb-learning.ts e kb-quality.ts.	Filtra o que merece virar treino. Reduz ruído, repetição e aprendizado errado.
SFT por trajetória	training-pack em trajectory-jsonl convertido para formato llama.cpp.	Ensina o modelo com exemplos de tarefa, raciocínio operacional e patch.
LoRA	llama.cpp finetune gera lora.bin.	Treina poucos parâmetros, mais leve para máquina local e reversível por workspace.
Adapter registry	forge-adapters.json.	Permite aprovar, ativar, trocar e auditar versões de aprendizado.
Hybrid fallback	orchestrator.ts.	Forge tenta local; se não consegue, escala para CLI premium mantendo contexto e rastreo.
Undo-aware learning	Fluxo de undo e qualidade.	Mudança desfeita não deve virar verdade aprendida.

4. O que é salvo

O treino é composto por dados locais e auditáveis. A ideia é permitir explicar para o cliente exatamente o que o modelo está usando para aprender.

Artefato	Local	Conteúdo
ai_training_examples	SQLite local	Exemplos curados com status, score, fonte, metadados e payload de trajetória.
training-pack-*	~/.orkestral/workspaces/<workspaceId>/learning/	Pasta exportada para treino com split de treino, validação e rejeitados.

train.trajectory.jsonl	training-pack-*	Trajетórias aprovadas para treino.
validation.trajectory.jsonl	training-pack-*	Holdout para avaliação antes de aceitar adapter.
rejected.trajectory.jsonl	training-pack-*	Itens descartados e seus motivos.
train.llama.txt	training-pack-*	Conversão textual compatível com llama.cpp finetune.
forge-training-jobs.json	workspace/learning	Histórico dos jobs de treino, status, progresso e erro.
lora.bin	workspace/learning/forge-train-*/	Adapter treinado localmente.
forge-adapters.json	workspace/learning	Registro dos adapters aprovados/ativos.
train.log	workspace/learning/forge-train-*/	Log bruto do processo de treino local.

5. Como o modelo aprende

1. O usuário executa tarefas reais no workspace, como alterar um componente, corrigir bug, atualizar contrato frontend/backend ou rodar QA.
2. A execução é capturada como trajetória: objetivo, contexto, arquivos, ações, ferramentas, validação e resultado.
3. A camada de qualidade decide se a trajetória é útil para treino. Um exemplo precisa ter escopo claro, mudança verificável, vínculo com source/repo e resultado confiável.
4. Trajetórias aprovadas entram no banco e depois são exportadas em packs. Trajetórias ruins ficam em rejected.trajectory.jsonl para auditoria.
5. O runner local chama llama.cpp finetune e treina um adapter LoRA em cima do modelo Forge GGUF.
6. Quando o job termina com sucesso, o adapter é registrado. Se estiver configurado, ele vira o adapter ativo daquele workspace.
7. Na próxima execução local, o runtime carrega o modelo base e injeta o loraPath. A resposta do Forge passa a ser influenciada pelo pós-treino local.

Ponto essencial

O modelo não “aprende” porque leu a Knowledge Base. Ler KB é RAG. Ele aprende quando trajetórias boas viram gradiente de treino em um adapter LoRA. Esse adapter é o artefato de aprendizado.

6. Critérios de qualidade do treino

A arquitetura evita transformar qualquer conversa ou resultado em treinamento. O princípio é: só entra no treino o que tem valor operacional durável.

Critério	Exemplo do que passa	Exemplo do que rejeita
Objetivo claro	“Adicionar phone ao RegisterFormData e enviar no submit”.	Mensagem genérica sem ação verificável.
Escopo de source	Execução ligada ao repo frontend, backend, mobile ou source específico.	Learning solto sem repo dono.
Mudança observável	Arquivos alterados, diff capturado, validação ou plano QA.	Resumo sem arquivos, sem resultado ou sem evidência.
Verificação	Tarefa concluída, validada ou marcada como not_applicable com justificativa.	Tarefa bloqueada, falha ou revertida.
Undo-aware	Mudança mantida pelo usuário.	Mudança desfeita por undo; não deve contaminar o treino.
Não repetição	Convenção nova e reutilizável.	Learning duplicado ou variação textual do mesmo ponto.

7. RAG vs Fine-tuning

Camada	O que faz	Quando usar
RAG / Knowledge Base	Recupera trechos, decisões e documentos relevantes em tempo de execução.	Quando o modelo precisa lembrar do estado atual do repo, APIs, páginas, componentes e regras.
Embeddings	Calcula similaridade para achar contexto relevante.	Quando há muitas páginas de KB e precisamos buscar por significado, não por palavra exata.
Fine-tuning LoRA	Altera o comportamento do modelo por exemplos treinados.	Quando há padrões repetidos de execução, estilo de patch, formato de resposta e decisões de arquitetura que o Forge deve

		internalizar.
Hybrid CLI	Escala tarefas que o local ainda não resolve bem.	Quando a tarefa exige raciocínio complexo, tool-calling mais robusto ou quando o Forge falha ao aplicar patch.

8. Execução local com adapter

Durante uma task, o orquestrador consulta se existe adapter ativo para o workspace. Se existir, ele envia esse caminho para o patcher local. O runtime node-llama-cpp cria o contexto do modelo com o LoRA carregado. O log registra lora=on para observabilidade.

Registro	getActiveForgeAdapterPath(workspaceId) em forge-local-training.ts.
Injeção	local-patcher.ts repassa loraPath para llamaChat.
Runtime	llama-runtime.ts chama model.createContext({ lora: loraPath }).
Trace	orchestrator.ts registra “Forge local usará adapter treinado”.

9. O que ainda falta para maturidade máxima

A base já caracteriza fine-tuning real, mas ainda existe uma diferença entre “pipeline funcional” e “sistema de treino maduro de produção”. Para chegar no nível ideal, os próximos gates devem ser obrigatórios.

Próximo gate	Por que é necessário
Avaliação automática pós-treino	Rodar validation.trajectory.jsonl e só ativar adapter se melhorar qualidade/latência/patch success.
Versionamento e rollback de adapter	Permitir voltar para adapter anterior se uma versão piorar comportamento.
Métricas por workspace	Medir taxa de sucesso local, fallback premium, economia de sessão, tempo médio e regressões.
Dataset balancing	Evitar que um repo, agente ou tipo de tarefa domine o treino.
Treino incremental controlado	Agrupar novidades relevantes sem retreinar por ruído

	pequeno.
QA de segurança	Nunca treinar segredos, tokens, dumps, logs sensíveis ou código privado fora do workspace local.

10. Definição técnica final

Hoje podemos descrever o Orkestral Forge assim:

Definição

O Orkestral Forge possui um pipeline local de aprendizado composto por RAG semântico para contexto, curadoria intrínseca de trajetórias verificadas, exportação de training packs e fine-tuning LoRA local via llama.cpp. O aprendizado persistente é materializado como adapter por workspace e carregado no runtime local, com fallback híbrido para CLI premium quando necessário.

Leitura honesta para produto

- Já não é apenas “memória” ou “busca similar”. Existe pós-treino local real por adapter.
- Ainda não é um modelo autônomo perfeito que dispensa Claude/Codex em todos os casos. O caminho correto é aumentar a taxa de sucesso local com datasets bons e avaliação rígida.
- O diferencial para o dev é a combinação: contexto do repo sempre atualizado, aprendizado local auditável, execução híbrida e transparência visual de quando Forge ou CLI foi usado.

Referência externa de implementação

A implementação de treino local segue a linha do utilitário llama.cpp finetune para GGUF com saída LoRA, conforme documentação pública de uso do finetune em modelos GGUF. A arquitetura do Orkestral adapta essa técnica para packs por workspace, registry de adapters e runtime local com LoRA ativo.

Glossário rápido

Termo	Significado no Orkestral
GGUF base	Arquivo do modelo Forge local baixado para a máquina do cliente.
LoRA adapter	Arquivo leve treinado que ajusta comportamento sem regravar o modelo base.
Training pack	Pacote local com dados aprovados, validação e rejeitados.
Trajectory	Registro estruturado de uma execução real de tarefa.

RAG	Busca de conhecimento em tempo de execução.
SFT	Treino supervisionado por exemplos de entrada e saída.
RFT-ready	Trajетória já estruturada para evoluir no futuro para reward/feedback training.